From Crochet Net to Puffy Polyhedron: Designing N-gon Animals with a Stretchy-Mesh Relaxation Tool

Tiffany Inglis

Abstract

Motivation

African flower crochet animals are low-resolution polyhedral models that are allowed to stretch. Crochet patterns usually include large diagrams showing how various polygons join together, along with photographs of the 3d objects taken from multiple angles. For designers, it's hard to prototype. Paper is too stiff and fabric requiring sewing and stuffing. For crafters, it's hard to keep track of where you are in the pattern.

In this paper, we seek to bridge the gap by first studying such crochet animals others have designed, come up with a mathematical representation, then use physical simulations to relax and inflate these models.

Field Study

I analyzed many African flower crochet animal patterns. All of them involve making regular n-gons then joining them via edges. They can be represented as polyhedral nets.

Structures typically considered degenerate (e.g. two squares) or impossible (e.g. two pentagons and a square) are valid structures with non-zero volume due to the flexibility of the material. Self joins (a polygon joined to itself) are also allowed.

All the examples are closed polyhedra with a clear inside and outside, although they could be temporarily open while under construction.

The models (giraffe, hippo, horse, turtle) use n-gons with $3 \leq n \leq 8$. The number of polygons range from 19 (turtle) to 42 (hippo). Animal models have "skinny" features like legs or necks, in contrast to standard polyhedra.

Representing a Stretchy Polyhedron

Conceptually a stretchy polyhedra consists of a set of polygons with edges that join them. To build it, we first assign n distinct vertices to each n-gon (in a counterclockwise fashion such that the Right Hand Rule produces an outward normal, defining its orientation). As one polygon join to another, their shared vertices are deduped. Polygons sharing an edge must also be oriented consistently (which means the shared edge must be defined in opposite directions).

Since our goal is to inflate the object, the polyhedral needs to be subdivided further via triangulation so the resolution is high enough to represent a rounded object. To this end, we first subdivide each base n-gons into n triangles by fanning out from the centroid. Then subsequent refinements involve dividing each triangle into four by joining edge bisectors.

To give it flexibility, we use a spring model, where each edge of the polyhedron is a spring. As it's being inflated, outward normal forces act upon faces (cross product of the edges makes the forces proportional to the area, which is desirable) while the edge springs pull towards their rest lengths. We collect these forces at vertices and step forward in time using the Forward Euler Method. If convergence is reached, the final state represents the stable inflated geometry.

Inflation algorithm

Before applying the inflation, we need to first establish an initial configuration. If we just use a random set of points as the vertices, the springs will initially contract/expand too rapidly, causing instability. So instead, we use the spring model with a small time step to relax the model first, much like putting a balloon in its rest, deflated state.

Next we apply the inflation as an outward pressure on the faces. To keep the pressure evenly distributed, it's scaled by area. Densely triangulated models take a long time to converge due to the number of vertices, so we use a hierarchical annealing process that alternates between inflating and subdividing. At each level, we inflate until convergence then subdivide the mesh (doubling the vertices each time). Three levels are enough to reach a sufficient smooth-looking model.

When the pressure is too low, the animals often end up with inverted heads or feet. This can be fixed by using a high pressure initially to "pop" them out, then gradually lowering the pressure at later stages to reach the desired level of inflation. Since numerical instability occurs at high pressures, we introduce a cubic hardening term to the linear springs for stability. Physically, this mimics the real-life behaviour of balloons reaching stability under pressure rather than growing unbounded.

For each level, we want choose the pressure that allows the model to stretch up to some amount (RMS stretch percent). Since this depends on a variety of factors, the easiest way is to just experiment with different values until you find something good enough. We run up to 8 trials for pressure estimation (up to 5000 steps), then take the best one and run it to completion if it hasn't converged already.

The 8 trials for pressure estimation only needs to happen for Level 1, the coarsest inflation. For subsequent levels, each time the resolution doubles, the pressure needed also doubles (from empirical evidence). So we can extrapolate from Level 1 results, and massively speed up the estimation process.

The algorithm is split into two stages.

Stage 1: Spring Relaxation

First triangulate the polyhedron. Then assign each vertex a random position. Use a basic spring model and iterate until the points are reasonably well-separated. This stage is akin to finding a relaxed deflated state.

Stage 2: Model Inflation

Now we're ready to inflate. A cubic hardening term is added to the linear springs to simulate real-life behaviour of balloons reaching a stable size under pressure than growing unbounded.

In practice, choosing a good pressure to inflate with is difficult. With too low a pressure, the model does not fully inflate, and some features (e.g. head and feet) get stuck inside. If the pressure is too high, convergence takes a long time and the shape is too puffy.

The whole mesh is advanced with many small explicit time steps, rather than jumped to a linearized solution (which can converge to a spurious, physically unreachable equilibrium), until the worst vertex's own displacement per step falls below a convergence tolerance.

Calibration

How much pressure to use is not directly meaningful across different nets, since the pressure needed scales with a net's total surface area rather than its vertex or face count. A calibration search instead lets the user pick a target *RMS edge stretch* (how far, on average, edges are allowed to stretch beyond their flat rest length) and automatically finds the pressure that reaches it, seeded from an empirical relationship between pressure and rest area and refined by a small number of relaxation trials.

Subdivision

A converged mesh can be refined further by quartering each triangle (splitting at edge midpoints) and re-relaxing at the new, finer resolution, carrying forward the shape already achieved rather than starting over from random noise. Repeated subdivide-then-inflate cycles progressively smooth out the piecewise-flat appearance of a coarse net into a more rounded solid, closer to what stuffed, stretchy fabric will actually look like.

Complication 1: Duplicate Edges

Complication 2: Self Joins

Complication 3: Unstable Constructions

- open tubes - degenerate/flat pieces - floppy pieces

The Interactive Viewer

The main application presents each net alongside its live 3D inflation: clicking *Relax* runs Stage 1, *Inflate* runs the calibrated Stage 2 relaxation toward a chosen target stretch, and *Subdivide* refines the mesh, with a *Run Simulation* button chaining all of these automatically through to a final resolution level. A companion flat-net panel always shows the corresponding unfolded 2D layout for reference, either derived automatically from the mesh's adjacency or, if the net was authored interactively, exactly as it was laid out by hand. A debug mode adds mesh-validity checks and face/vertex normal overlays for diagnosing orientation problems while designing a new net.

Summary and Conclusions

net2polyhedron supports the full pipeline described above — authoring, relaxing, calibrated inflation, and incremental subdivision — for a family of test solids (the Platonic solids) and several hand-designed animal nets, and includes an automated test suite covering the geometry and solver code. Remaining work includes translating an inflated digital net back into row-by-row crochet instructions for each panel, and validating the simulated shapes against physically crocheted and stuffed prototypes.

Acknowledgements

References